

REPORT BUILDER

Data Model

DynamoDB Table, S3 Stores & Report_Design Reference

Overview

This document describes the data model behind the Report Builder. It is a companion to the technical walkthrough and the API specification, giving the concrete record shapes, key patterns, and indexes the implementation will use.

The design follows a single-table DynamoDB pattern. Reports, per-report assistant conversation history, and run records all live in one table, distinguished by partition key (PK) and sort key (SK) patterns and scoped to the effective portal-user identity. Larger blobs are kept out of DynamoDB: versioned report-design snapshots and run result CSVs live in S3, referenced by key.

Throughout, <effectiveld> is the effective portal-user identity - the Admin_Override email when an override is present, otherwise the signed-in user's email. It scopes every item, so a cross-user reference simply returns nothing.

Reading the key patterns

Values in angle brackets are substituted at runtime, e.g. REPORT#<reportId> becomes REPORT#01J2... . Records that share a partition key (a user's reports, their conversation messages, and their runs) are stored together and retrieved with owner-scoped queries - the core idea behind single-table design.

Stores at a glance

STORE	HOLDS	NOTES
DynamoDB single table	Report, Conversation message, and Run items	PK/SK, PAY_PER_REQUEST, two GSIs; all items scoped by USER#<effectiveld>
S3 - design snapshots	Versioned JSON snapshot of each Report_Design on save	blockPublicAccess=ALL, versioned, SSE; off the read path (history/restore)
S3 - Result_Store	Run result CSVs	blockPublicAccess=ALL, versioned, SSE; a disabled lifecycle rule reserves expiry

DynamoDB single table

One table holds three item types, all scoped by the effective identity. The name GSI drives the My Reports A-Z/Z-A sort and case-insensitive search; the recency GSI drives the most-recent-50 run list.

Table: `BrytReportBuilder` (single table)

Report

PK

USER#<effectiveId>

SK

REPORT#<reportId>

Stores the serialised Report_Design inline (small JSON, well under the 400 KB item limit) plus metadata for the My Reports list. Update on an existing reportId overwrites in place - no duplicate.

ATTRIBUTE	TYPE	DESCRIPTION
reportId	String	ULID; stable across saves
name	String	1-100 chars on save (1-200 while editing)
description	String	(optional) 0-500 chars
tables	List	Table names used, for the My Reports badges
design	Map	Serialised (canonicalised) Report_Design JSON
schemaVersion	Number	Report_Design schema version for forward migration
createdAt / updatedAt	String	ISO 8601 timestamps
GSI1PK / GSI1SK	String	USER#<effectiveId> / NAME#<lowername> (sort + search)

Conversation message

PK USER#<effectiveId> **SK** REPORT#<reportId>#MSG#<ts>

Per-report assistant history, time-ordered. Converse is stateless, so history is passed from this store each turn; reopening a report restores it.

ATTRIBUTE	TYPE	DESCRIPTION
reportId	String	Report this message belongs to
ts	String	ISO 8601 timestamp (ordering)
role	String	user or assistant
content	String	Message text
appliedChanges	List	(optional) Per-change descriptions for an applied edit

Run

PK USER#<effectiveId> **SK** REPORT#<reportId>#RUN#<runNo>

One per asynchronous execution. Recency GSI returns the most-recent 50 newest-first.

ATTRIBUTE	TYPE	DESCRIPTION
reportId	String	Report this run belongs to
runNo	Number	Sequential run number within the report
trigger	String	manual (scheduled is out of scope)
status	String	Queued Running Complete Failed Cancelled
startedAt	String	ISO 8601 timestamp
rowCount	Number	(optional) 0..999,999,999 on Complete
errorMessage	String	(optional) <= 1000 chars on Failed
resultLocation	String	(optional) Result_Store key, only when Complete + verified
GS12PK / GS12SK	String	REPORT#<reportId> / STARTED#<ts> (recency list)

Global Secondary Indexes

INDEX	GSI PK	GSI SK	ENABLES
GSI1 (name)	USER# <effectiveId>	NAME# <lowername>	My Reports A-Z / Z-A sort and case-insensitive name search
GSI2 (run recency)	REPORT# <reportId>	STARTED#<ts>	List the up-to-50 most-recent runs for a report, newest first

The serialised Report_Design

The Report_Design is a logical description of intent - selected tables, columns, joins, filters, and sort order - never SQL. It is the single shared model edited by both the flow canvas and the assistant, and it is stored inline on the Report item as canonicalised JSON.

Round-trip identity is guaranteed by treating tables, columns-per-table, joins, and filters as sets (order-insensitive, deduplicated on write) and sort as an ordered list. Serialise is a canonicalised JSON encoding (sorted keys, sorted set members); deserialise validates against the catalog and manifest and rehydrates. A design referencing a disallowed table, column, or join is rejected without touching the persisted copy.

- › tables - each an allow-listed Catalog table with its selected allow-listed columns
- › joins - each references a Join_Manifest join id, with left/right table names
- › filters - column + operator + bound parameter value(s); never string-concatenated into SQL
- › sort - an ordered list of column + direction rules (left-to-right precedence)
- › scope - optional single Bryt_Number narrowing, always re-checked as a member of the authorised set
- › schemaVersion - for forward migration of the serialised form

What lives in S3 (not DynamoDB)

Two kinds of larger data are kept in S3 and referenced from DynamoDB by key, keeping items small and cheap to query while the bulkier content sits in object storage. Both buckets block all public access, are versioned, and are server-side encrypted.

- › Report-design snapshots - snapshots/<effectiveld>/<reportId>.json, a versioned JSON snapshot on each save, off the read path (history/restore)
- › Run result CSVs - results/<effectiveld>/<reportId>/<runNo>.csv, written by the pipeline and delivered via a short-lived pre-signed URL
- › A disabled S3 lifecycle rule on the Result_Store reserves CSV expiry so it can be enabled later with no data-model change

A note on scope

These record shapes come from the design document and are intended to convey intent and structure. Exact attribute names, optionality, and any additional GSIs may be refined during implementation without changing the overall approach.